twoday

AI HACKATHON

Agentic Engineering using GitHub Copilot

Today's speakers

Hans Olav Sunde

Global Technology Director
Global Digital Engineering

hans.sunde@twoday.com

Thomas Martinsen

Global Principal Advisor
Global Digital Engineering

thomas.martinsen@twoday.com

Niklas Gustafsson

Global Senior Engineer
Global Digital Engineering

niklas.gustafsson@twoday.com

Simo Roikonen

Global Principal Architect
Global Digital Engineering

simo.roikonen@twoday.com

Søren Thestrup

Solution Lead
Data & AI, Denmark

soren.thestrup@twoday.com

The way we deliver and operate is in change

Agentic **design** and **control** in practice

AI-native **development** agents

These agents are specialized for software development tasks. They range from assisted coding to automated development workflows, integrating AI directly into how software is built and maintained.

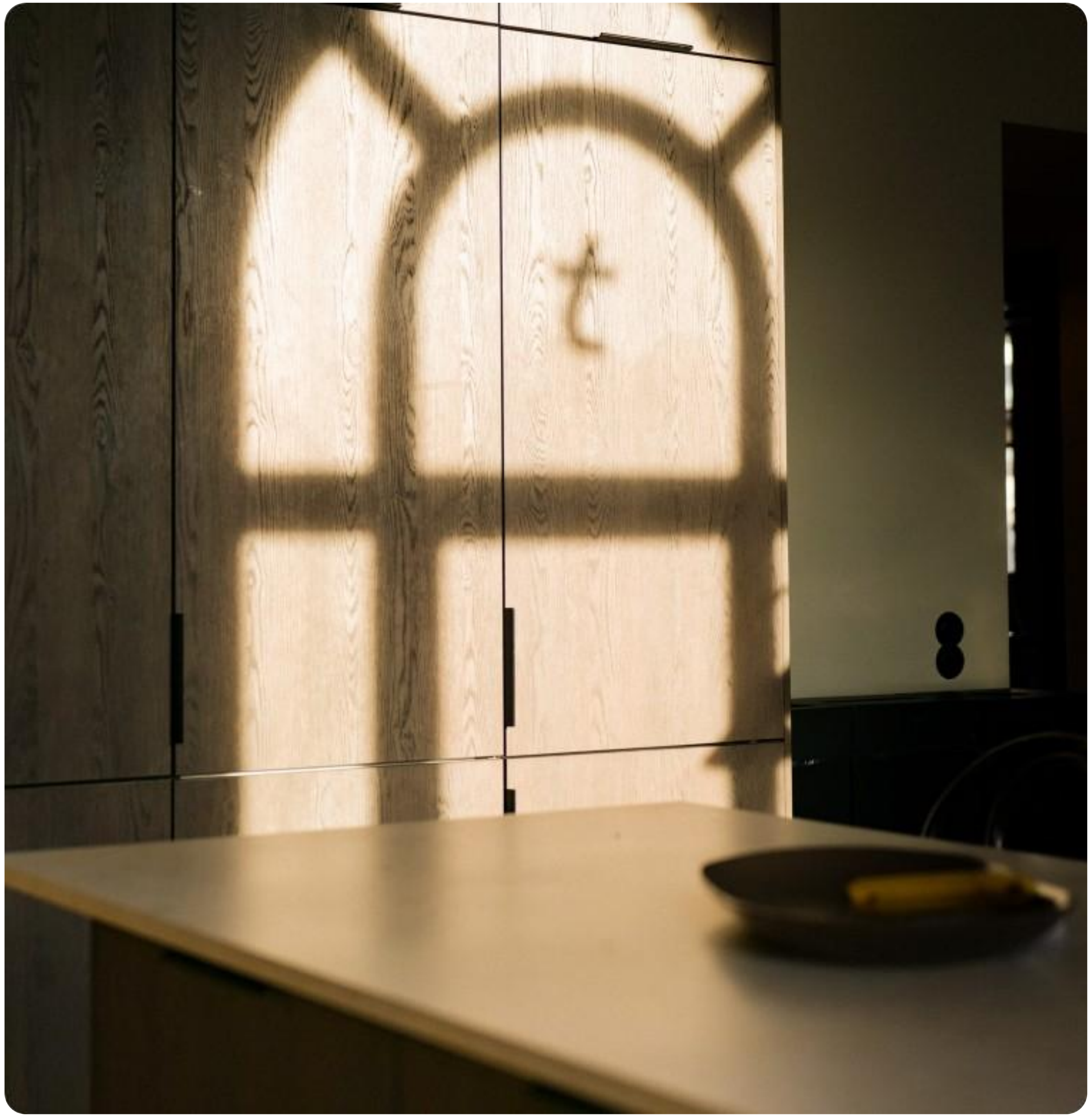
Assistant and **interactive** agents

These agents support users through chat or interfaces. They are reactive and enhance human decision-making and execution across domains.

Autonomous and **process** agents

These agents operate with increasing levels of autonomy, ranging from structured workflows to more dynamic, goal-oriented behavior. They act across systems with limited or no human intervention.

Introduction to Agentic Engineering



How development changes

MANUAL CODING

- Developer writes everything
- Manual scaffolding
- Hand-written tests
- Large code surface

AI-ASSISTED DEVELOPMENT

- Developer writes the code
- AI code suggestions
- Prompt-driven scaffolding
- AI-assisted refactoring

AGENT-DRIVEN DEVELOPMENT

- Developer defines intent
- Spec-driven planning
- Agent execution
- Human validation gates

Before

Manual coding

Producer → Intent designer + reviewer

Now

Agent-driven development

Different levels of agentic engineering

Prototype, proof-of-concepts or explorative solutions with no focus on security

Build using vibe-coding tools like Lovable, GitHub Spark, Figma Make, etc.

Pilots, MVPs, or small internal tools with minimum focus on scalability and governance

Build using pro-coding tools combined with agentic tools like GitHub Copilot, Claude Code, etc. and using structured knowledge, workflows, and reusable skills

Structured, predictable, scalable, and governed enterprise solutions

Build using pro agentic tools combined using structured knowledge, workflows, reusable skills, and enterprise governance patterns

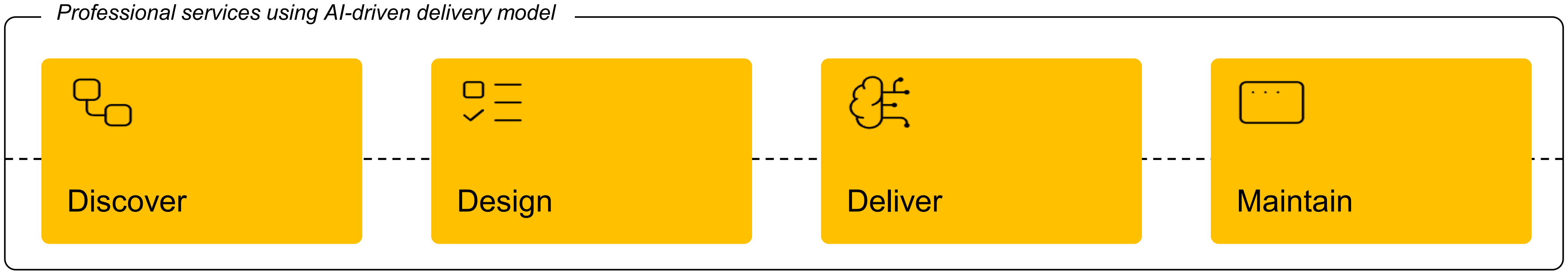
Not recommended for production but great for testing ideas, working on different workflow, A/B testing. Can be done by most people in the organization without any technical knowledge including knowledge about enterprise security, governance, etc.

Ready for production but with limited focus on security, governance, and scalability. Build using structured knowledge, reusable components and skills. Development is handled by agents but validated by developers. Optimal for small to medium greenfield and brownfield projects.

Ready for production with extensive focus on security, governance, and scalability. Build using structured knowledge, workflows, reusable components, and skills. Development is handled by agents but validated by developers. Optimal for medium to large greenfield and brownfield projects with mixed tech stack including modernization.

Delivery of enterprise software solutions using an agent-driven approach

Roles
Innovation lead, Discovery facilitator, UX specialist, AI engineer, QA approver, Project orchestrator



Deliver using agent-driven development

Structured, AI-orchestrated processes where the developer defines intent, specifications guide planning, agents execute tasks, and human validation gates ensure control and quality across the end-to-end flow. Designed to be predictable, reusable, team-scalable, and governed.

Discover — Design — **Deliver** — Maintain

DEVELOPMENT FLOW

Getting started

Enable agent-driven development structure and flow to software project.

Structuring knowledge

Describe infrastructure, audience, security model, tech stack, architecture.

Engineering primitives

Add agents, instructions, skills, and prompts to the solution.

Compile

Spec-driven development

Use spec engine to work on features and tasks and follow spec-driven principals.

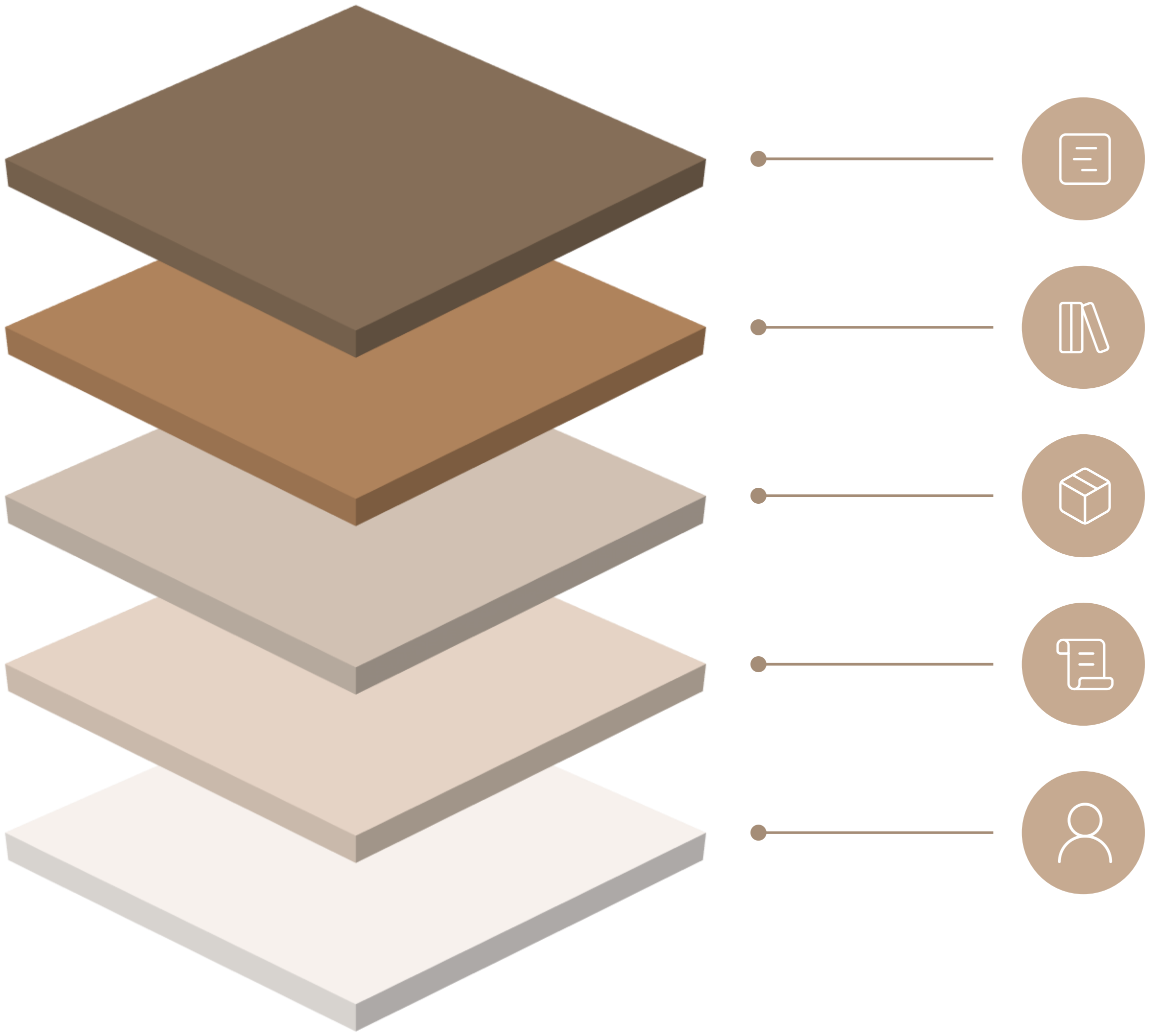
Compile into standard formats

Compile knowledge and primitives into standard formatted artifacts.

Orchestrate and review changes

Use human orchestration and review, and AI test and review

Onboarding a new **member** to the team



Tools – to be able to operate, and to execute on actions, the member will need access to tools and applications.

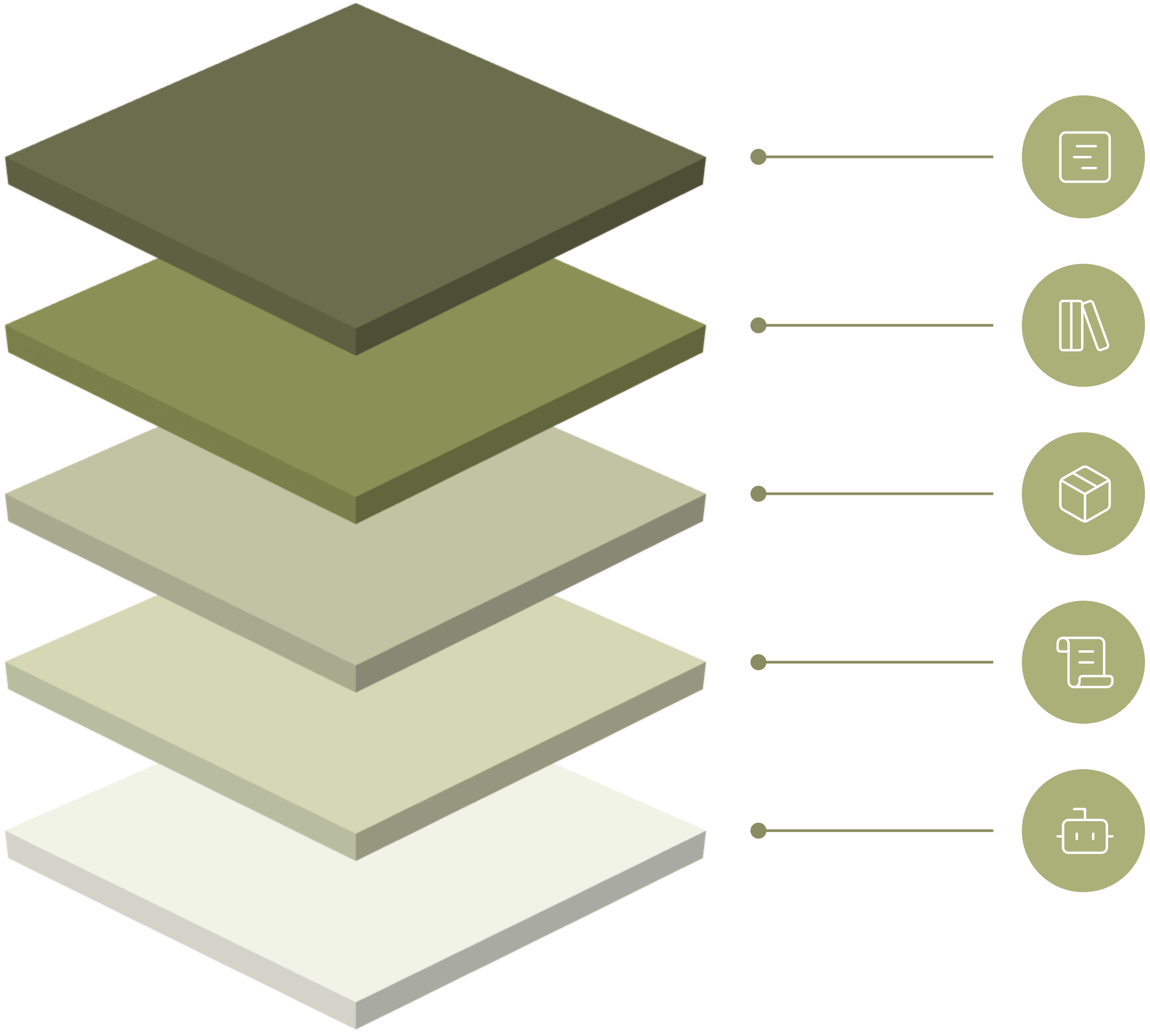
Information – the member will need access to the basic information about the company, industry, the project, etc.

Playbooks – when working on projects the member will need guidelines and playbooks on how things are done.

Rules – to be able to work in best possible way, the member needs to understand the rules and best practices for the team.

Role – a new member to the team will have a role on the team; frontend developer, architect, tester, project manager, etc.

Onboarding a new **agent** to the team



Tools – the MCP tools extending the grounded knowledge in the models ensuring that the agents have the newest knowledge.

Models – the large language model used to understand requirements and how to implement them based on primitives.

Skills – the agent should have access to catalog of skills specifying how to implement topics like frontend, architecture, etc.

Instructions - the agent should have access to instructions on how to implement specific features in the project.

Agents – custom agent with a specified role on the team; frontend agent, architect agent, reviewer agent, tester agent, etc.

Spec-driven development

Follow spec-driven principals: Change => Implement => Archive.
Use human orchestration and review, and AI test and review

Discover | Design | Deliver | Maintain

SPEC-DRIVEN DEVELOPEMENT

- Feature specification
- Planning and context
- Implementation
- Validation and test
- Integration

Approve and review **specification**

Approve and review **architecture**

Approve and review **code**

Approve and review **test**

Agentic building blocks

Primitives are the core configurable elements that developers iteratively refine to ensure reliable outcomes.

AGENTIC BUILDING BLOCKS

Engineering primitives

Add agents, instructions, skills, and prompts to the solution.

Agents

Reusable agents that orchestrate the role, permissions and operating boundaries.

Instructions

Scoped rules and the standards that the agent must follow and in which context.

Skills

Capabilities the agent can use within specific areas and context windows.

Hooks

Capabilities the agent can use within specific areas and context windows.

Orchestration

Use spec engine to work on features and tasks and follow spec-driven principals.

Specifications

Source of truth covering intent, constraints and acceptance criteria.

Memory

Decisions, notes and state maintained and carried forward.

MCP tools

Use spec engine to work on features and tasks and follow spec-driven principals.

Plugins

Decisions, notes and state maintained and carried forward.

Predictable, reusable, and scale across teams

Development time decrease with

+50%

Quality and security up with

+20%

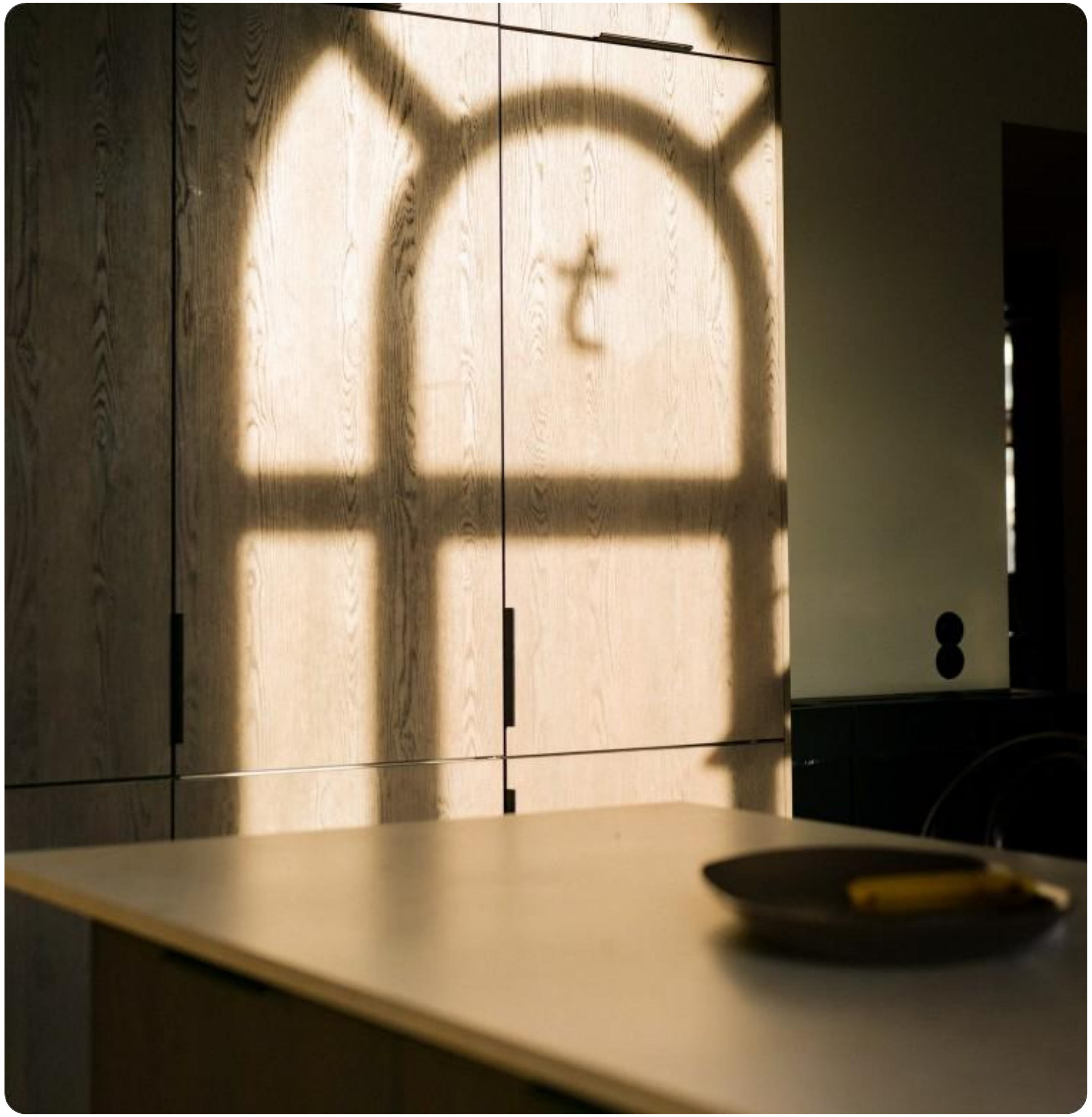
Improve standardization

>50%

Happy developers

+20%

Introduction to GitHub Copilot



Stay on top of your AI tools and resources

AI ORCHESTRATION

Visual Studio Code / GitHub Copilot app

GitHub Copilot* / Claude Code / Codex

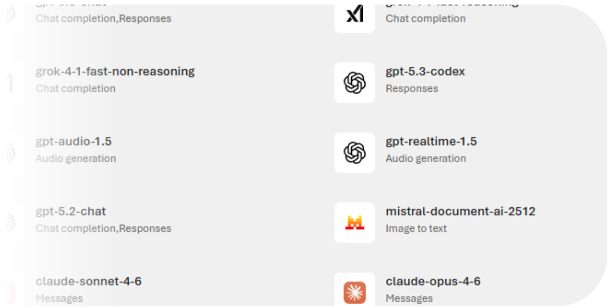
OpenSpec / GitHub SpecKit

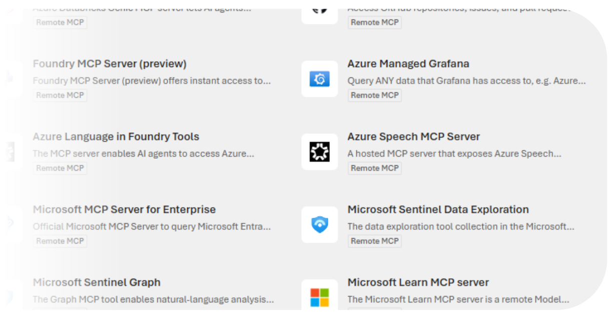
SERVERS AND MODELS

MCP servers (Azure, Learn, GitHub)

Models for planning

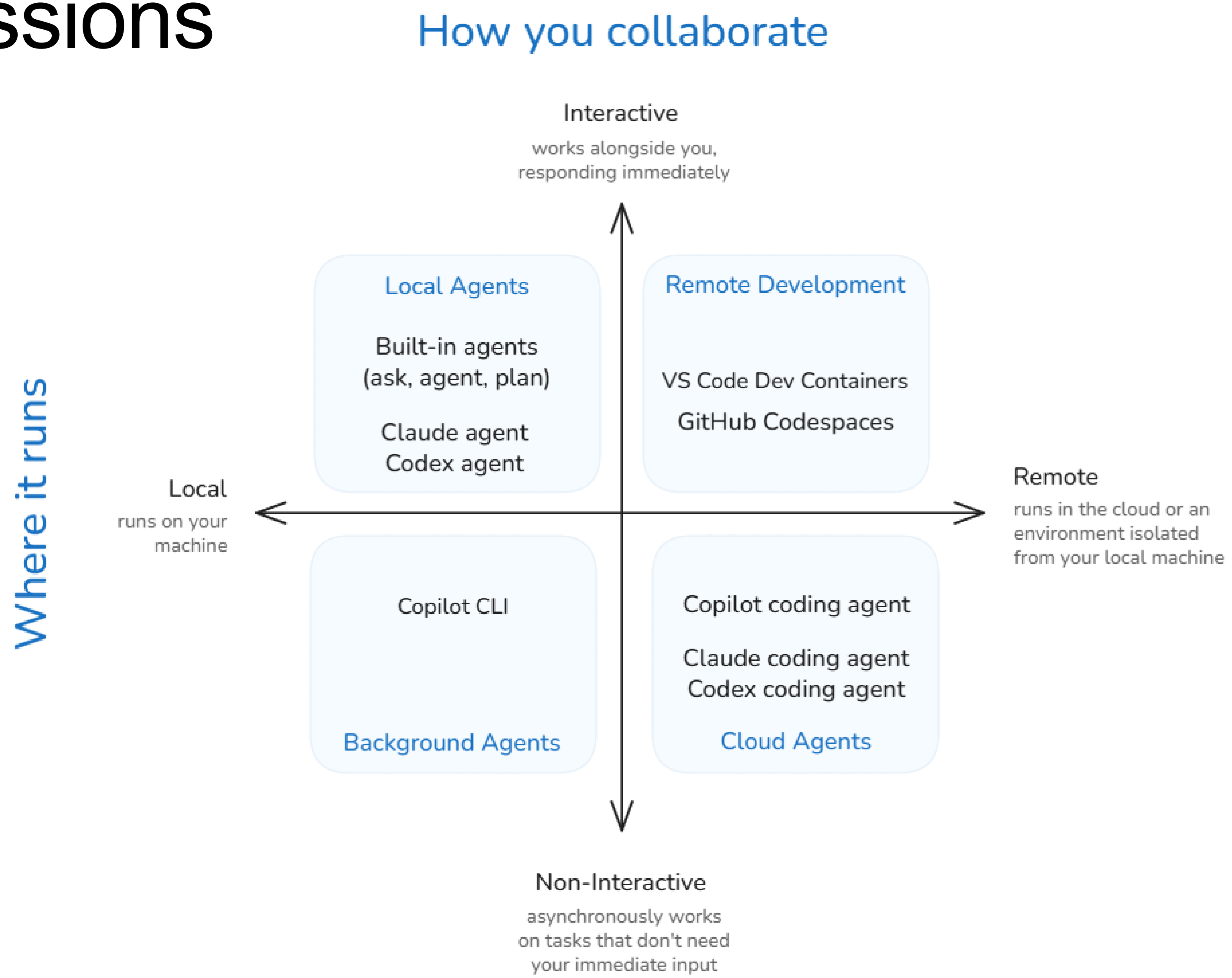
Models for implementation



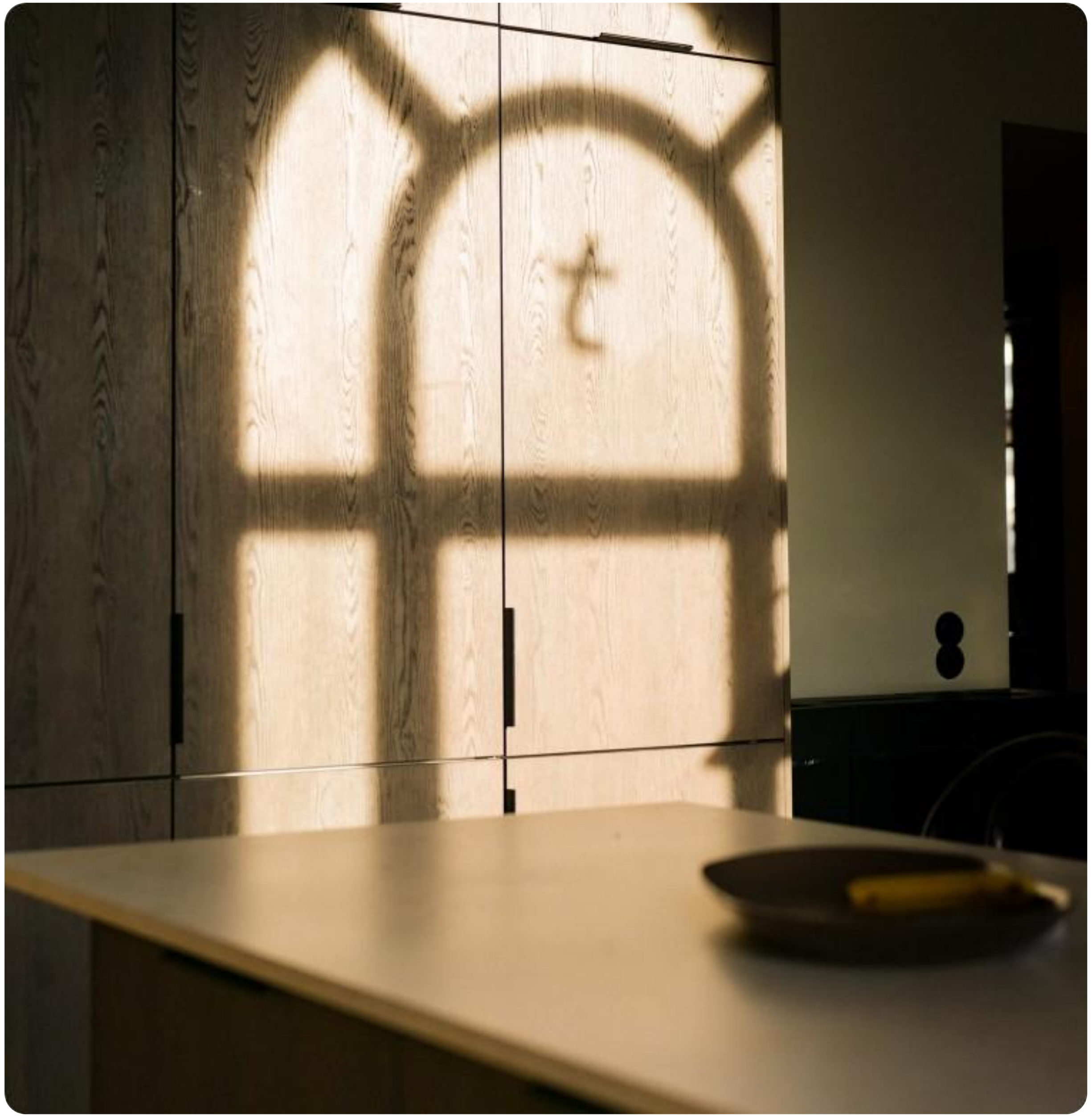


* Business (300 PRPM) or Enterprise (1500 PRPM)

Agent sessions



Agentic Incident Service case



https://github.com/thomasmartinsen/event-hackathon-20260528

Agentic Incident Service – case topics

GitHub
https://github.com/thomasmartinsen/event-hackathon-20260528

Agentic Incident Service

- Reliable agent-driven development
- Orchestrator efficiency
- Testing as an evaluation primitive
- Security and resilience

twoday